

COURSE CODE

SBT-DF202

COURSE TITLE

Computer and Digital Forensics

LAB TITLE

Steganography and Hidden Data Detection

NAME

Kumpon Emmanuel Nandom

STUDENT ID

2025/FWSD/11325

DATE

05/09/2026

PLATFORM USED

Kali Linux

INSTRUCTOR

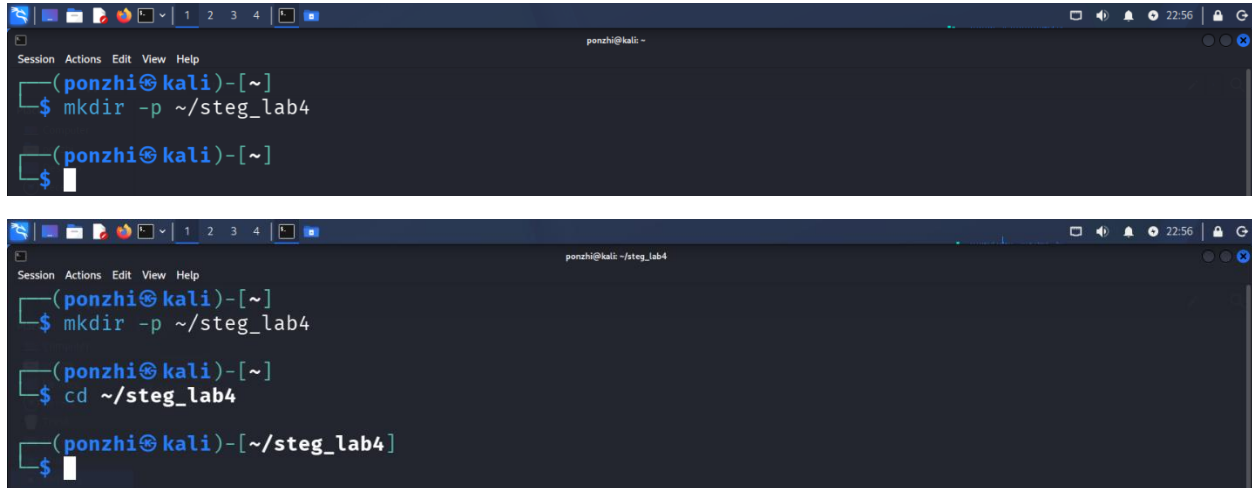
Mr. Aminu Idris, AMCPN
Commandant

LAB OVERVIEW

By completing this lab, I have demonstrated the ability to explain the difference between steganography and encryption, describe insertion and substitution-based hiding techniques including least significant bits (LSB), prepare a carrier image and secret evidence file, use Steghide to embed and extract concealed data, calculate and compare cryptographic hashes, perform a controlled password dictionary test using StegSeek, and document all activities in a repeatable forensic report.

Setup

`mkdir -p ~/steg_lab4` and `cd ~/steg_lab4`

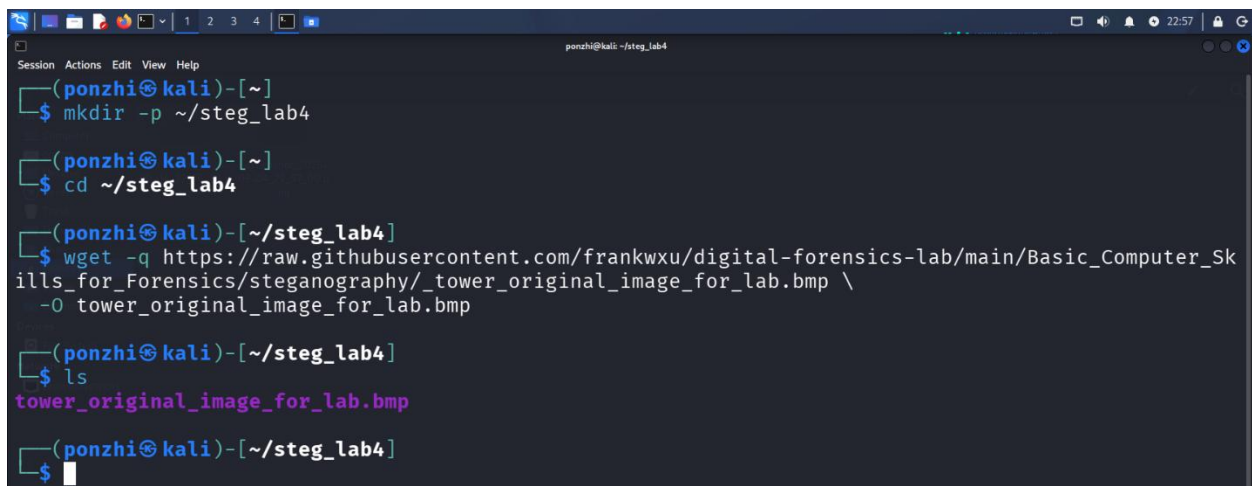


The first screenshot shows a terminal window with the prompt `(ponzhi@kali)-[~]`. The user enters `$ mkdir -p ~/steg_lab4`, and the prompt returns. The second screenshot shows the terminal window with the prompt `(ponzhi@kali)-[~]`. The user enters `$ cd ~/steg_lab4`, and the prompt changes to `(ponzhi@kali)-[~/steg_lab4]`.

Part A — Prepare and Preserve the Evidence

Download the authorised lab carrier image

Using the source image referenced in the training material, since lab4.docx doesn't have a direct URL



The screenshot shows a terminal window with the prompt `(ponzhi@kali)-[~/steg_lab4]`. The user enters `$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp \ -O tower_original_image_for_lab.bmp`. The prompt returns. The user then enters `$ ls`, and the output is `tower_original_image_for_lab.bmp`.

Confirm the file type

```
ponzhi@kali: ~/steg_lab4
Session Actions Edit View Help
(ponzhi@kali)-[~]
$ mkdir -p ~/steg_lab4

(ponzhi@kali)-[~]
$ cd ~/steg_lab4

(ponzhi@kali)-[~/steg_lab4]
$ wget -q https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Basic_Computer_Skills_for_Forensics/steganography/_tower_original_image_for_lab.bmp \
-O tower_original_image_for_lab.bmp

(ponzhi@kali)-[~/steg_lab4]
$ ls
tower_original_image_for_lab.bmp

(ponzhi@kali)-[~/steg_lab4]
$ file tower_original_image_for_lab.bmp
tower_original_image_for_lab.bmp: PC bitmap, Windows 3.x format, 1365 x 2265 x 24, resolution 2834 x 2834 px/m, cbSize 9277494, bits offset 54

(ponzhi@kali)-[~/steg_lab4]
$
```

Calculating and recording the SHA-256 hash of the ORIGINAL image

```
ponzhi@kali: ~/steg_lab4
Session Actions Edit View Help
(ponzhi@kali)-[~/steg_lab4]
$ sha256sum tower_original_image_for_lab.bmp | tee original_image_sha256.txt
6508bebb94bd8ec0833b9f855580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp

(ponzhi@kali)-[~/steg_lab4]
$
```

```
ponzhi@kali: ~/steg_lab4
Session Actions Edit View Help
(ponzhi@kali)-[~/steg_lab4]
$ sha256sum tower_original_image_for_lab.bmp | tee original_image_sha256.txt
6508bebb94bd8ec0833b9f855580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp

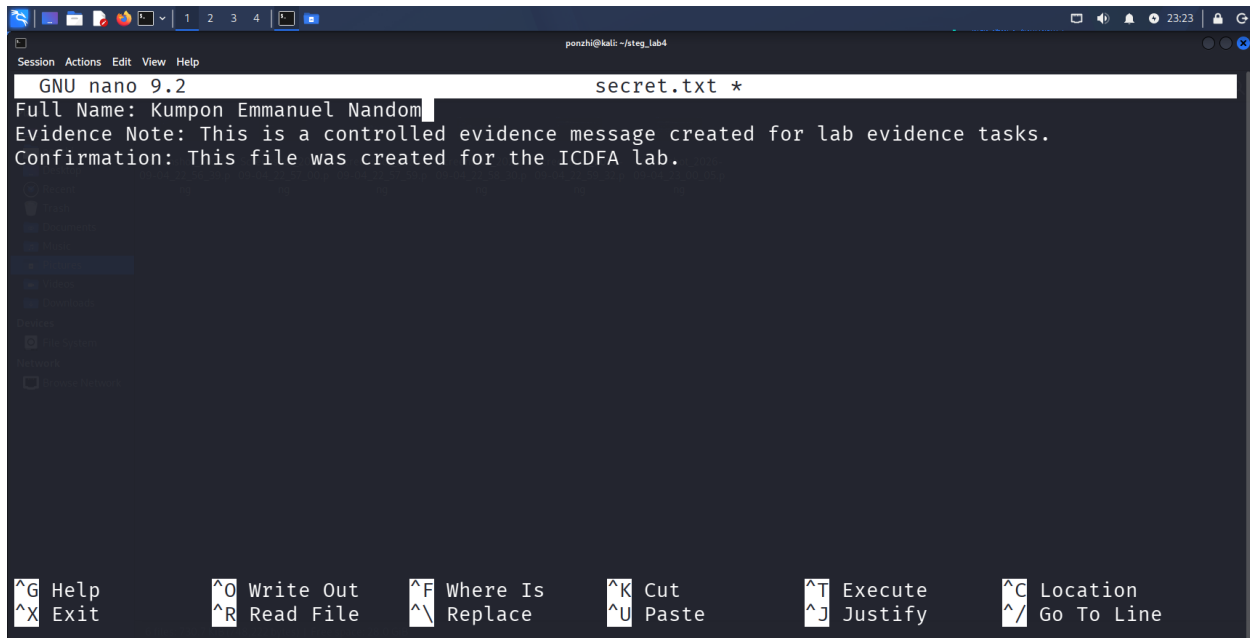
(ponzhi@kali)-[~/steg_lab4]
$ ls
original_image_sha256.txt tower_original_image_for_lab.bmp

(ponzhi@kali)-[~/steg_lab4]
$ cat original_image_sha256.txt
6508bebb94bd8ec0833b9f855580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp

(ponzhi@kali)-[~/steg_lab4]
$
```

Part B — Create the Secret Evidence File

Create a text file named secret.txt



The screenshot shows the nano 9.2 text editor interface. The title bar indicates the file is 'secret.txt *'. The main editing area contains the following text:

```
Full Name: Kumpon Emmanuel Nandom
Evidence Note: This is a controlled evidence message created for lab evidence tasks.
Confirmation: This file was created for the ICDFA lab.
```

The bottom status bar displays various keyboard shortcuts: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^F Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify, ^C Location, and ^_ Go To Line.

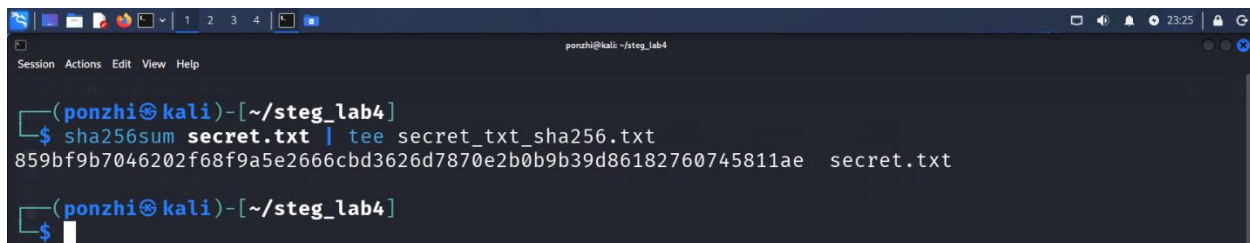


The screenshot shows a terminal window with the following commands and output:

```
(ponzhi@kali)-[~/steg_lab4]
$ nano secret.txt

(ponzhi@kali)-[~/steg_lab4]
$ cat secret.txt
Full Name: Kumpon Emmanuel Nandom
Evidence Note: This is a controlled evidence message created for lab evidence tasks.
Confirmation: This file was created for the ICDFA lab.
```

Hash the secret file



The screenshot shows a terminal window with the following commands and output:

```
(ponzhi@kali)-[~/steg_lab4]
$ sha256sum secret.txt | tee secret_txt_sha256.txt
859bf9b7046202f68f9a5e2666cbd3626d7870e2b0b9b39d86182760745811ae  secret.txt
```

Part C — Embed the Hidden File with Steghide

Create a **separate** stego image (-sf) so the original stays untouched, and use Steghide to embed secret.txt into a **new stego image**

```
ponzhi@kali: ~/steg_lab4
$ steghide embed -ef secret.txt -cf tower_original_image_for_lab.bmp \
-sf tower_stego_lab4.bmp -p 1234
embedding "secret.txt" in "tower_original_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab4.bmp" ... done

ponzhi@kali: ~/steg_lab4
$
```

Hash the resulting stego image

```
ponzhi@kali: ~/steg_lab4
$ steghide embed -ef secret.txt -cf tower_original_image_for_lab.bmp \
-sf tower_stego_lab4.bmp -p 1234
embedding "secret.txt" in "tower_original_image_for_lab.bmp" ... done
writing stego file "tower_stego_lab4.bmp" ... done

ponzhi@kali: ~/steg_lab4
$ ls
original_image_sha256.txt  secret_txt_sha256.txt  tower_stego_lab4.bmp
secret.txt                tower_original_image_for_lab.bmp

ponzhi@kali: ~/steg_lab4
$ sha256sum tower_stego_lab4.bmp | tee stego_image_sha256.txt
5018ed538842aa056e0f35e2a0bba27acac4f065e3d447aa10a4db66339796fb tower_stego_lab4.bmp

ponzhi@kali: ~/steg_lab4
$
```

Confirm that the original wasn't touched

```
ponzhi@kali: ~/steg_lab4
writing stego file "tower_stego_lab4.bmp" ... done

ponzhi@kali: ~/steg_lab4
$ ls
original_image_sha256.txt  secret_txt_sha256.txt  tower_stego_lab4.bmp
secret.txt                tower_original_image_for_lab.bmp

ponzhi@kali: ~/steg_lab4
$ sha256sum tower_stego_lab4.bmp | tee stego_image_sha256.txt
5018ed538842aa056e0f35e2a0bba27acac4f065e3d447aa10a4db66339796fb tower_stego_lab4.bmp

ponzhi@kali: ~/steg_lab4
$ sha256sum tower_original_image_for_lab.bmp
Command 'sha256sum' not found, did you mean:
  command 'sha256sum' from deb coreutils
Try: sudo apt install <deb name>

ponzhi@kali: ~/steg_lab4
$ sha256sum tower_original_image_for_lab.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp

ponzhi@kali: ~/steg_lab4
$
```

Should match original_image_sha256.txt

Part D — Extract and Verify the Hidden Evidence

Create a directory named Extracted

```
ponzhi@kali: ~/steg_lab4
$ mkdir -p extracted
$
```

Extract the concealed file using the password 1234

```
ponzhi@kali: ~/steg_lab4
$ mkdir -p extracted

ponzhi@kali: ~/steg_lab4
$ steghide extract -sf tower_stego_lab4.bmp -xf extracted/extracted_secret.txt -p 1234
wrote extracted data to "extracted/extracted_secret.txt".

ponzhi@kali: ~/steg_lab4
$
```

Verify integrity

Both hashes should be identical, confirming lossless extraction

```
ponzhi@kali: ~/steg_lab4
$ mkdir -p extracted

ponzhi@kali: ~/steg_lab4
$ steghide extract -sf tower_stego_lab4.bmp -xf extracted/extracted_secret.txt -p 1234
wrote extracted data to "extracted/extracted_secret.txt".

ponzhi@kali: ~/steg_lab4
$ diff secret.txt extracted/extracted_secret.txt 66 echo "Files match - no differences"
Files match - no differences

ponzhi@kali: ~/steg_lab4
$ sha256sum secret.txt extracted/extracted_secret.txt
859bf9b7046202f68f9a5e2666cbd3626d7870e2b0b9b39d86182760745811ae secret.txt
859bf9b7046202f68f9a5e2666cbd3626d7870e2b0b9b39d86182760745811ae extracted/extracted_secret.txt

ponzhi@kali: ~/steg_lab4
$
```

Part E — Compare Original and Stego Images

List Files with sizes

```
ponzhi@kali: ~/steg_lab4
$ ls -l tower_original_image_for_lab.bmp tower_stego_lab4.bmp
-rw-rw-r-- 1 ponzhi ponzhi 9277494 Sep  4 22:57 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 ponzhi ponzhi 9277494 Sep  4 23:27 tower_stego_lab4.bmp
```

Hashes files

```
ponzhi@kali: ~/steg_lab4
$ ls -l tower_original_image_for_lab.bmp tower_stego_lab4.bmp
-rw-rw-r-- 1 ponzhi ponzhi 9277494 Sep  4 22:57 tower_original_image_for_lab.bmp
-rw-rw-r-- 1 ponzhi ponzhi 9277494 Sep  4 23:27 tower_stego_lab4.bmp

ponzhi@kali: ~/steg_lab4
$ sha256sum tower_original_image_for_lab.bmp tower_stego_lab4.bmp
6508bebb94bd8ec0833b9f85580cda4261d89632ddc89e7f9f7093b5e653c4db tower_original_image_for_lab.bmp
5018ed538842aa056e0f35e2a0bba27acac4f065e3d447aa10a4db66339796fb tower_stego_lab4.bmp
```

Initial bytes using xxd. Header comparison

```
ponzhi@kali: ~/steg_lab4
$ xxd tower_original_image_for_lab.bmp | head -9
00000000: 424d 3690 8d00 0000 0000 3600 0000 2800  BM6.....6 ... (.
00000010: 0000 5505 0000 d908 0000 0100 1800 0000  ..U.....
00000020: 0000 0000 0000 120b 0000 120b 0000 0000  .....
00000030: 0000 0000 0000 1e22 3520 2338 1e20 381f  ...."5 #8. 8.
00000040: 203a 2425 3f28 2945 2729 482a 2b4d 2a2c  :$%?( )E' )H**M*,
00000050: 4e29 2e4f 282e 5127 3052 2831 5328 3254  N).O(.Q'0R(1S(2T
00000060: 2f39 5b35 4163 3d4e 6f44 5979 465f 8147  /9[5Ac=NoDYyF_.G
00000070: 6687 4669 8b45 6889 4468 8645 6785 4a69  f.Fi.Eh.Dh.Eg.Ji
00000080: 8842 5f7e 3b57 7637 4e6e 3243 6432 3e60  .B_~;Wv7Nn2Cd2>`
```



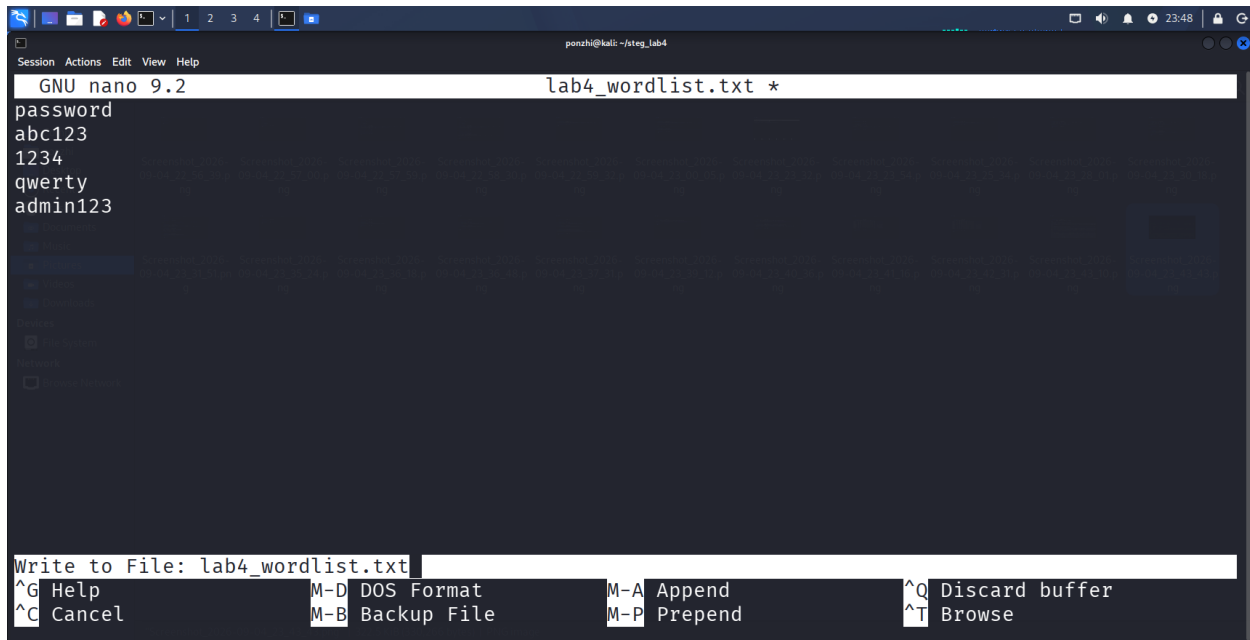
```
ponzhi@kali: ~/steg_lab4
$ strings tower_stego_lab4.bmp | head -20
"5 #8
:$$%( 'E')H**M*,N).O(.Q'0R(1S(2T/9[5Ac=NoDYf_
B_~;Wv7Nn2Cd2>`1:\17Z79\79\;;_;;_:=`79\67]56\56\67]67]56\56\56\79\57Z46Y57Z46Y68[68[35X56X23U23U45W
23U12T34V34V56X45W34V-.P./Q,-O/0R/0R32R10P/.N.-M/.N/.N..L--K//M++I..L--K,-I,-I**G**G),H6)E6)E'*F6*C
'+D)*D')A(*B6(@')A')A6)>6)>'*$'<+,A+,A*+@(>)'(='(=>(>)=(');%'9#%7#%7#%7"$6$686)8$'6%%5%%5#3##3
$$4!!1!/
#!#+-/??BGMPNPNQJLLFHHHJJIKKJLLNNIKKBDD?AA;:<?>@EDF?>@=>?>@;: <=> : ≤ 578: ≤ 6899;<9;>@ABDEJNOKOPMO
PLNOIJNJKOORVRUYRWZY^adkn\svrx}msxZ^cJNS/47$),
#7<?afiotwux|ruyuvzdeiXY]MNRONR]\`fei\[_GHLHIMNOSRSW00UIIOEEKEEKPUQVRVQTXWZ^`chdgladi^afRRXMMSSV[Y
\arUY\_cmptilp`cgX[_VY]_bf`cg\_cafifknkothlqaej0SXfJ0BfK8>C7=B:@E:@E9BF8AE9BF=FJ<DK7?F8AE:CG<EI:CG9
BF8AE9BF;DH:BI9AH9AH9AH;CJ<DK9AH8@G7AH7AH8BI7AH7AH8BI7AH7AH<FM:DK:DK8BI;EL:DK8BI7AH@HO>FM;CJ;CJ?GN?
GN:BI9AH?GN<DK<DK<DK=EL>FM=EL<DK=HL?JN>IM8CG9DH?JN?JN;FJ@KO>IM<GK<GK<GK9DH;FJ?JN@KO=HL>IM@KOJUYZeif
quoz~q|
kvzR]a?JNALPCNRBMQGPTAJNAJNFOSGPTENRALP@KOBMQALPCNRALP>IMBMQEPTFQUEPTFQUGRVEPTTEPTGRVEPT@KOCMTCMTDNU
CMTCMTEOVEOVDNUGQXEOVFPWFPWAKRBLSGQXHRYPWPWGQXISZFPWDNUFPWGQXGQXISZFPWHQZIR[FOXFOXIR[IR[KU\ISZISZISZ
HRYFPWCLUHRYHRYHRYHRYHRYGQXHRYPJT[FPWHRYISZGQXISZISZFPWHRYISZJT[GQXISZFPWGQXGQXJT[JT[JT[ISZJT[
JT[HRYISZJS\JS\IR[GPYFOXGPYIR[IR[JS\IR[FOXFOXHQZIR[LU^LU^JS\JS\JS\HQZGPYJS\JS\HQZIR[IR[IR[HQZHQZHQZ
HQZHQZJS\HQZGPYGPYGPYHQZHQZENWHQZGPYHQZGPYFOXGPYGPYFOXIR[GPYGPYFOXDMVFOXGPYENWIR[FOXFOXHQZHQZHQZHQZ
HQ[FOXBKTAJSLUFOXHQZFOXDMVFOXENWFOXGPYGPYHQZHQZGPYIR[HQZGPYGPYHQZJS\KT]KT]KT]LU^LW_JU]GRZHS[IT\HS[
KT]GPYJS\KT]JS\KT]LU^JS\MV_KT]JS\JS\IR[IR[JS\JS\JU]IT\JU]IT\IT\KV^KV^IT\KV^IT\HS[HS[HS[IT\KV^MX`KV^
```

Explain why two images may look visually similar while their underlying digital content and hashes are different.

When embedding data, it only changes the least-significant bits of pixel data. This means that the two BMP images look the same to human eyes, as we can't notice a tiny change in color. However, when we calculate the SHA-256 hash, even a single flipped bit will give a completely different hash. The file size and header information remain mostly the same because Steghide does not alter the image dimensions; it only changes the pixel data.

Part F — Controlled Password Dictionary Test

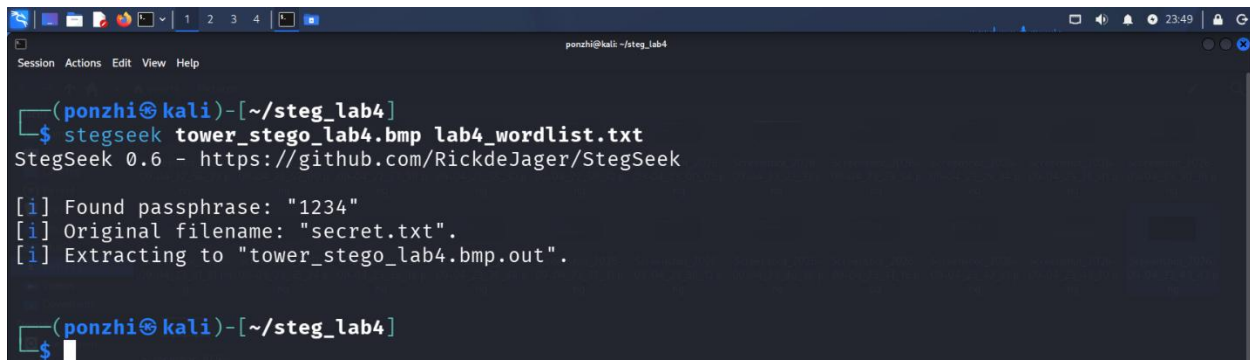
Build a small controlled wordlist that includes the known password



```
GNU nano 9.2 lab4_wordlist.txt *
password
abc123
1234
qwerty
admin123

Write to File: lab4_wordlist.txt
^G Help      M-D DOS Format  M-A Append     ^O Discard buffer
^C Cancel    M-B Backup File M-P Prepend    ^T Browse
```

Run StegSeek against the stego image



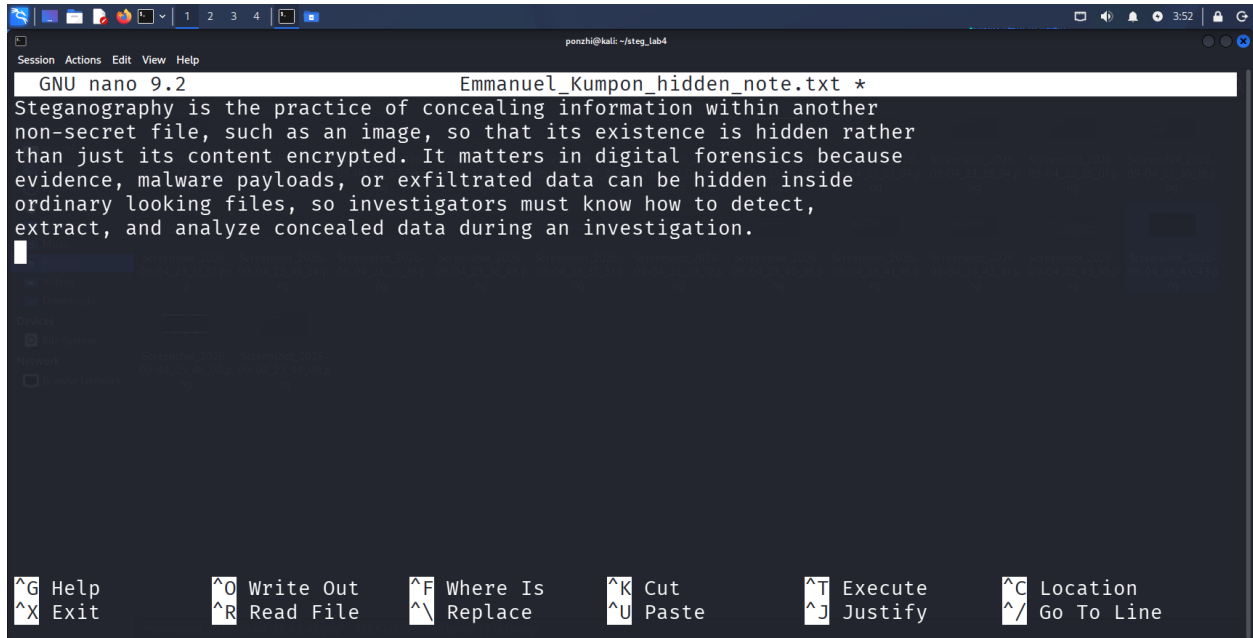
```
(ponzhi@kali)~/steg_lab4
$ stegseek tower_stego_lab4.bmp lab4_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1234"
[i] Original filename: "secret.txt".
[i] Extracting to "tower_stego_lab4.bmp.out".

(ponzhi@kali)~/steg_lab4
$
```

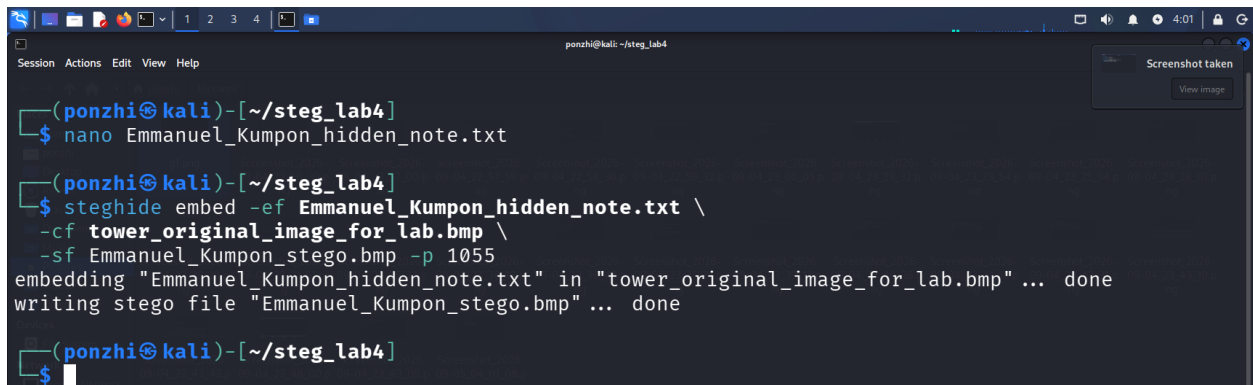
Part G — Independent Student Task

Create your own evidence file



```
GNU nano 9.2 Emmanuel_Kumpon_hidden_note.txt *
Steganography is the practice of concealing information within another
non-secret file, such as an image, so that its existence is hidden rather
than just its content encrypted. It matters in digital forensics because
evidence, malware payloads, or exfiltrated data can be hidden inside
ordinary looking files, so investigators must know how to detect,
extract, and analyze concealed data during an investigation.
```

Embed it with a password 1055

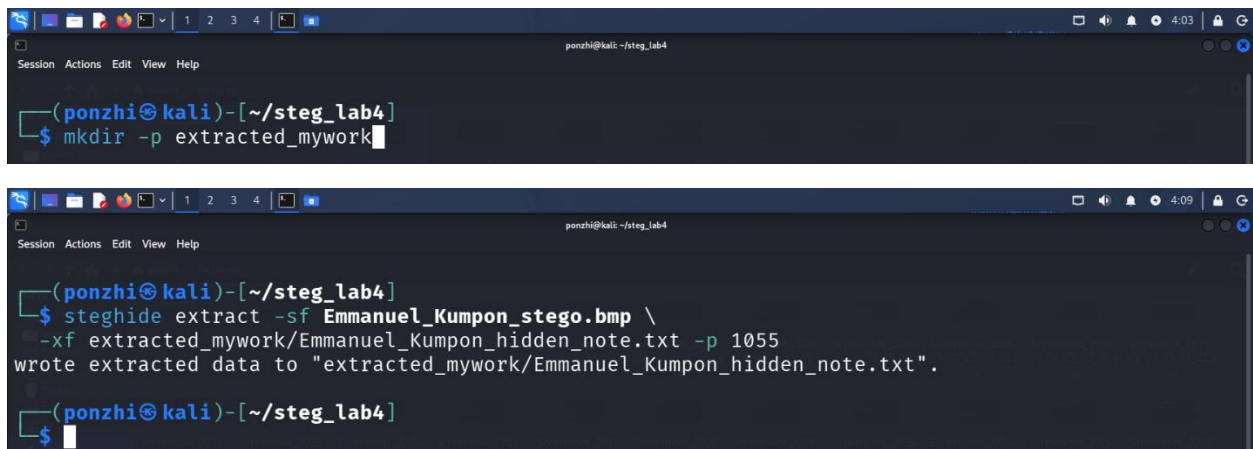


```
(ponzhi@kali)-[~/steg_lab4]
$ nano Emmanuel_Kumpon_hidden_note.txt

(ponzhi@kali)-[~/steg_lab4]
$ steghide embed -ef Emmanuel_Kumpon_hidden_note.txt \
-cf tower_original_image_for_lab.bmp \
-sf Emmanuel_Kumpon_stego.bmp -p 1055
embedding "Emmanuel_Kumpon_hidden_note.txt" in "tower_original_image_for_lab.bmp" ... done
writing stego file "Emmanuel_Kumpon_stego.bmp" ... done

(ponzhi@kali)-[~/steg_lab4]
$
```

Extract it again



```
(ponzhi@kali)-[~/steg_lab4]
$ mkdir -p extracted_mywork

(ponzhi@kali)-[~/steg_lab4]
$ steghide extract -sf Emmanuel_Kumpon_stego.bmp \
-xf extracted_mywork/Emmanuel_Kumpon_hidden_note.txt -p 1055
wrote extracted data to "extracted_mywork/Emmanuel_Kumpon_hidden_note.txt".

(ponzhi@kali)-[~/steg_lab4]
$
```

Verify integrity using hashes

```
ponzhi@kali: ~/steg_lab4
Session Actions Edit View Help

(ponzhi@kali)-[~/steg_lab4]
$ steghide extract -sf Emmanuel_Kumpon_stego.bmp \
  -xf extracted_mywork/Emmanuel_Kumpon_hidden_note.txt -p 1055
wrote extracted data to "extracted_mywork/Emmanuel_Kumpon_hidden_note.txt".

(ponzhi@kali)-[~/steg_lab4]
$ sha256sum Emmanuel_Kumpon_hidden_note.txt extracted_mywork/Emmanuel_Kumpon_hidden_note.txt
26284ca34e7b0efb96a4682b25aa4bb52f98cc966d4c1def102841b31f0b10ae Emmanuel_Kumpon_hidden_note.txt
26284ca34e7b0efb96a4682b25aa4bb52f98cc966d4c1def102841b31f0b10ae extracted_mywork/Emmanuel_Kumpon_hidden_note.txt

(ponzhi@kali)-[~/steg_lab4]
$
```

```
ponzhi@kali: ~/steg_lab4
Session Actions Edit View Help

(ponzhi@kali)-[~/steg_lab4]
$ steghide extract -sf Emmanuel_Kumpon_stego.bmp \
  -xf extracted_mywork/Emmanuel_Kumpon_hidden_note.txt -p 1055
wrote extracted data to "extracted_mywork/Emmanuel_Kumpon_hidden_note.txt".

(ponzhi@kali)-[~/steg_lab4]
$ sha256sum Emmanuel_Kumpon_hidden_note.txt extracted_mywork/Emmanuel_Kumpon_hidden_note.txt
26284ca34e7b0efb96a4682b25aa4bb52f98cc966d4c1def102841b31f0b10ae Emmanuel_Kumpon_hidden_note.txt
26284ca34e7b0efb96a4682b25aa4bb52f98cc966d4c1def102841b31f0b10ae extracted_mywork/Emmanuel_Kumpon_hidden_note.txt

(ponzhi@kali)-[~/steg_lab4]
$ diff Emmanuel_Kumpon_hidden_note.txt extracted_mywork/Emmanuel_Kumpon_hidden_note.txt 66 echo "
Match"
Match

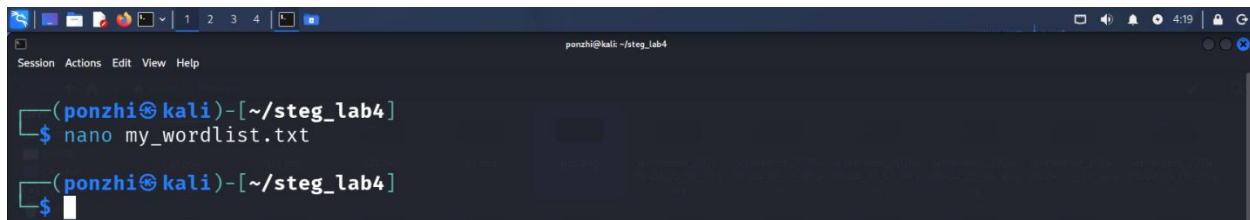
(ponzhi@kali)-[~/steg_lab4]
$
```

Create a small wordlist containing your chosen password

```
ponzhi@kali: ~/steg_lab4
Session Actions Edit View Help

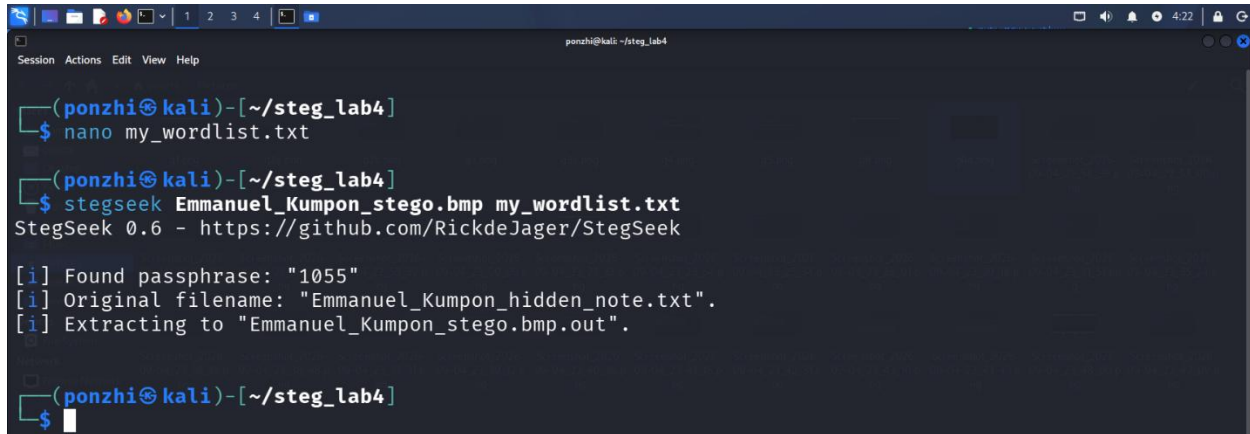
GNU nano 9.2 my_wordlist.txt *
test123
1055
football
sunshine
admin123

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```



```
(ponzhi@kali)-[~/steg_lab4]
$ nano my_wordlist.txt
(ponzhi@kali)-[~/steg_lab4]
$
```

Test the recovery workflow



```
(ponzhi@kali)-[~/steg_lab4]
$ nano my_wordlist.txt
(ponzhi@kali)-[~/steg_lab4]
$ stegseek Emmanuel_Kumpon_stego.bmp my_wordlist.txt
StegSeek 0.6 - https://github.com/RickdeJager/StegSeek

[i] Found passphrase: "1055"
[i] Original filename: "Emmanuel_Kumpon_hidden_note.txt".
[i] Extracting to "Emmanuel_Kumpon_stego.bmp.out".
(ponzhi@kali)-[~/steg_lab4]
$
```

Conclusion

This lab showed the full process of image-based steganography in digital forensics. It covered hiding data in a carrier file, checking its integrity, detecting its presence by comparison, and recovering it using a password-cracking method.

We used Steghide to embed secret.txt into tower_original_image_for_lab.bmp. This demonstrated how to hide a data file inside a normal-looking BMP image. We changed only the least-significant bits of the pixel data, which are small enough to be visually undetectable. Yet, these changes allowed Steghide to extract the hidden content, confirmed by matching SHA-256 hashes and a clean diff.

By comparing the original and the stego images, we reinforced a key forensic principle: file size and appearance can stay the same while the underlying data can be very different. That's why cryptographic hashing is a reliable way to detect tampering or hidden data, rather than visual checks.

Using StegSeek for a dictionary attack showed the limits of steganography as a security tool. We recovered a weak or common password (1234) from the rockyou.txt wordlist in seconds. This highlights that steganography only hides the presence of data, not the protection of its contents. The strength of the password is crucial in keeping hidden evidence away from prying eyes. The quicker StegSeek, replacing the slower StegCracker, also shows how fast forensic tools are evolving. Investigators need to keep up with the latest methods instead of relying on older, slower ones.

Completing the independent task in Part G helped us practice these steps without instructions. This confirmed that the workflow embeds, hashes, extracts, verifies, and attempts recovery—works with any file and password combination. It is a repeatable process that an investigator can apply to real forensic evidence with suspected steganographic content.